IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 3 July 2025, accepted 21 July 2025, date of publication 31 July 2025, date of current version 7 August 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3594619

RESEARCH ARTICLE

Ultralight Situation-Aware Hypervisor With Adaptive Starvation-Free Feedback Control for Embedded System

MINJUNG KIM[1], (Student Member, IEEE), AND DAEJIN PARK[2], (Member, IEEE)
[1]School of Electronic and Electrical Engineering, Kyungpook National University, Daegu 41566, South Korea
[2]School of Electronics Engineering, Kyungpook National University, Daegu 41566, South Korea

Corresponding author: Daejin Park (boltanut@knu.ac.kr)

This work was supported in part by the Brain Korea 21 4th (BK21 FOUR) Project under Grant 4199990113966; in part by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education, 10%, under Grant RS-2018-NR031059; in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) grant funded by Korean Government, Ministry of Science and ICT (MSIT), 30%, 30%, 30%, respectively under Grant 2022-0-01170, Grant RS-2023-00228970, and Grant RS-2025-02218227; and in part by the Electronic Design Automation (EDA) tool was funded by the IC Design Education Center (IDEC), South Korea.

ABSTRACT In recent years, microcontrollers are increasingly using hypervisors to achieve efficient and adaptable real-time performance in resource-constrained environments, enabling better resource management and system isolation. However, traditional hypervisor scheduling approaches often rely on static scheduling or require all scheduling decisions to be determined at compile time, limiting their ability to adapt to dynamic workload changes and real-time demands. To address these issues, this study introduces a scheduling technique that combines round-robin and priority scheduling. Rather than simply merging the two methods—which would also combine their disadvantages—the proposed approach resolves the fixed time-quantum issue of round-robin by dynamically adjusting the time-quantum based on priority for each round. In addition, to address the starvation problem in priority scheduling, a feedback control system is designed to maintain the average waiting time at an acceptable level. To mitigate the temporal overhead of determining time quanta, the design incorporates memory and functional isolation, significantly reducing context switching overhead. Our experimental results show a 54% reduction in context switching overhead compared to FreeRTOS, requiring only 44us for switching operations. In dynamic priority scenarios using computationally intensive dummy tasks to simulate real workloads, the system reduced the execution time by up to 38% for high priority conditions while appropriately deferring lower priority tasks. By improving resource utilization and responsiveness to external environmental changes, this approach enables robust and adaptive software execution, providing an effective solution for real-time embedded systems.

INDEX TERMS Dynamic scheduling, embedded optimization, feedback control, lightweight switching, task starvation, priority scheduling.

I. INTRODUCTION

In recent years, microcontroller units (MCUs) have evolved into essential components of complex electronic systems, demanding real-time performance, adaptability, and efficiency in resource-constrained environments. To address these challenges, advanced resource management techniques such as hypervisors have gained prominence. Widely used in domains such as automotive systems and cloud computing, hypervisors provide mechanisms to allocate and optimize hardware resources across multiple tasks and applications, ensuring efficient and reliable operation even in highly demanding environments [1], [2].

Although traditional hypervisor solutions often target full-scale operating systems in resource-rich environments, there is a growing need for specialized hypervisors that

The associate editor coordinating the review of this manuscript and approving it for publication was Fazel Mohammadi.

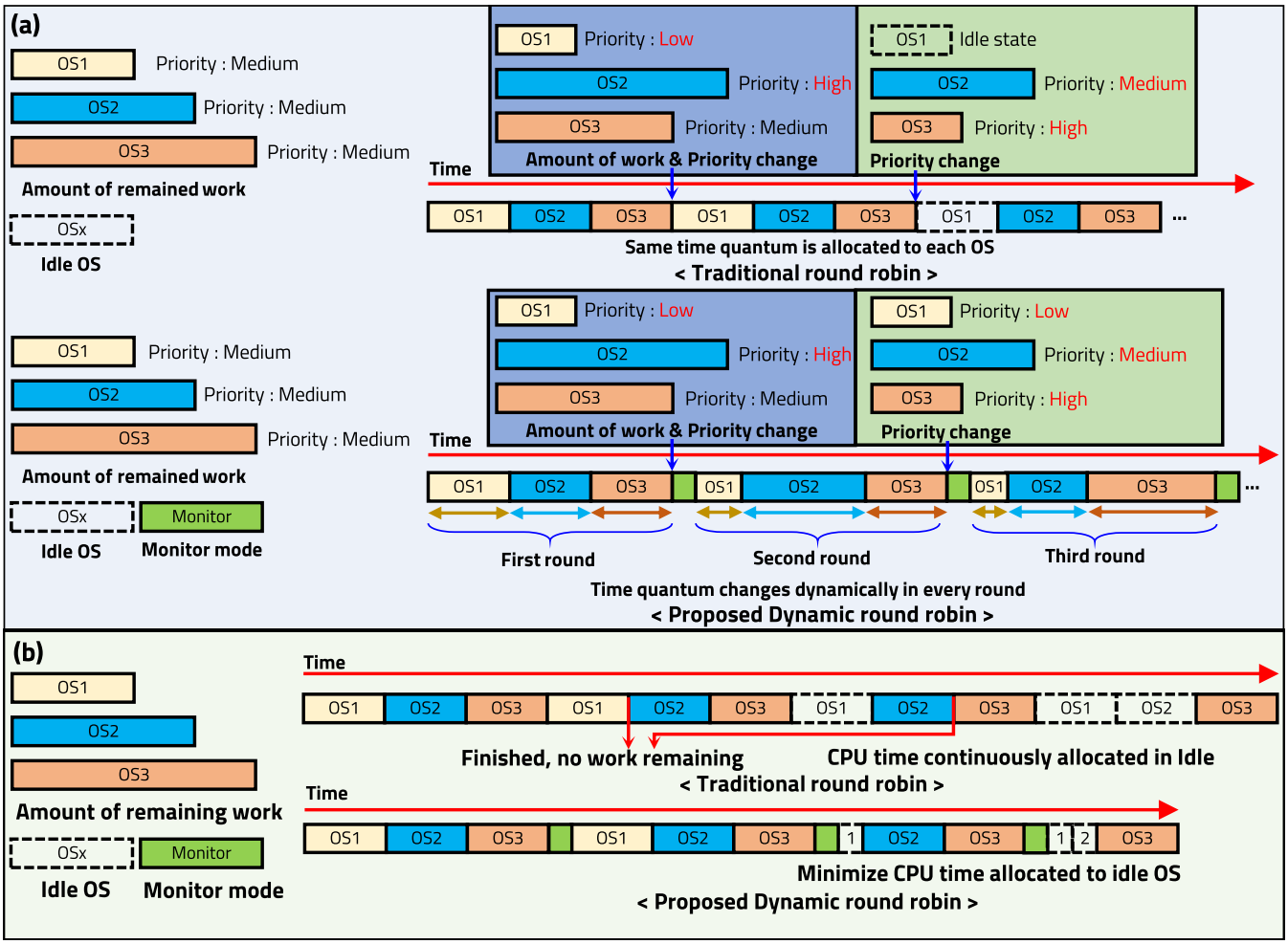


FIGURE 1. Comparison between traditional round-robin and dynamic round-robin (a) Response to external changes (b) Policies to ensure minimum execution time.

accommodate multiple lightweight operating systems on MCUs. Given strict constraints on computational and memory resources, these designs must minimize overhead and footprint while providing dynamic resource management. However, many existing hypervisors rely on static configurations and compilation-time scheduling decisions, making them unsuitable for embedded scenarios where external triggers, unpredictable I/O, and shifting workloads demand real-time adaptation.

Such resource constraints highlight the shortcomings of conventional hypervisor architectures. Large-scale hypervisors that depend on ARM Hypervisor mode or complex Memory Management Units (MMUs) often exceed what MCUs can accommodate, both in code size and runtime overhead. Moreover, their reliance on static resource allocation and narrow scheduling criteria can result in suboptimal performance when priorities change or new tasks emerge. Under these static schemes, memory fragmentation or prolonged starvation can also arise as tasks compete for limited resources, further undermining real-time performance. In a resource-limited MCU environment, this rigidity can lead to missed deadlines for critical tasks, inefficient resource usage, and ultimately compromise the reliability of the system.

In contrast, the bottom illustration in Figure 1 (a) illustrates how dynamic round-robin scheduling works in OS management. Through long-term observation of urgency patterns and event occurrences, the framework dynamically adjusts the scale factors that influence the final priority calculations, enabling more sophisticated priority management. In real-world scenarios, factors such as changes in the user's environment or unpredictable I/O interrupts can alter the priorities of OS or the remaining computational workload. To address these issues, this paper introduces a dynamic round-robin scheduling framework optimized for soft real-time embedded systems, where adaptability and responsiveness are critical. This framework adjusts time slices in real-time based on task priorities, allowing for more flexible execution while maintaining isolation between OS instances. The lightweight and independent execution model ensures robust handling of OS-level breaks while allocating more time to high-priority

tasks and minimizing allocation for lower-priority tasks. This adaptive approach improves efficiency and reliability.

Another issue with traditional round-robin scheduling is its inefficiency in handling idle OS instances. Even when there are no tasks left to execute, idle OS instances still receive unnecessary time slices due to the rigid allocation of fixed-time quanta. This approach wastes valuable time that could otherwise be allocated to OS instances with urgent execution needs, as illustrated in the top section of Figure 1 (b).

To address this inefficiency, our proposed dynamic round-robin approach optimizes time-quantum allocation. By minimizing the time-quantum assigned to idle OS instances, as shown in the bottom section of Figure 1 (b), more processing time is made available for actively running OS instances, significantly improving overall system performance.

Because the integration of priority-based scheduling with round-robin enhances resource allocation by considering task importance, it also inherits a key drawback of priority scheduling: starvation of lower-priority tasks. To address this limitation, our framework employs a feedback control mechanism that dynamically adjusts scheduling decisions based on the accumulated waiting times of tasks. By continuously monitoring and regulating average wait times across priority levels, this approach mitigates starvation risks while maintaining system responsiveness and fairness.

However, implementing this dynamic scheduling framework still presents challenges, such as the potential computational overhead from real-time adjustments and increased context switching. To address these issues, we propose an ultralight context switching mechanism. This streamlined process minimizes resource-intensive operations, allowing the system to maintain high responsiveness without compromising the benefits of dynamic scheduling.

In automotive MCUs, where bare metal firmware and manually coded interrupt service routines dominate, the lack of dynamic task management has been a significant limitation. This proposed framework bridges the gap by enabling advanced task prioritization and resource allocation without the complexity of a full-scale OS.

The main contributions of this paper are summarized as follows:

- A novel proportional integral derivative (PID)-controlled dynamic scheduling framework for adaptive, real-time adjustment of time quanta in resource-constrained systems.
- An average waiting time (AWT)-based feedback control mechanism to prevent task starvation and ensure system fairness.
- An ultralight context switching implementation that reduces overhead by 54% compared to FreeRTOS.

The effectiveness of the framework is evaluated through simulations of motor rotation speed-driven task adjustments on MCUs, focusing on efficiency, responsiveness, and reliability under various conditions. These results highlight its potential as a transformative approach to resource management in resource-constrained real-time environments. Although our evaluation focuses on automotive applications, this framework can be broadly applied to various domains where task priorities dynamically shift based on environmental conditions. This versatility demonstrates the framework's potential as a general-purpose solution for resource-constrained systems requiring adaptive task management.

The remainder of this paper is structured as follows. Section II discusses background and considerations for the proposed method. Section III presents our dynamic round-robin scheduling framework and implementation details. Sections IV and V describe the experimental setup and results, followed by conclusions in Section VI.

II. RELATED WORK

As round-robin scheduling is widely recognized, its limitations are also well documented. Consequently, dynamic round-robin approaches have been consistently explored as attempts to overcome these inherent shortcomings [3], [4], [5]. In addition, dynamic round-robin scheduling is utilized not only in local environments but also in various contexts. In particular, research on dynamic round-robin approaches has also been conducted in the field of networking [6]. Moreover, dynamic round-robin has frequently been employed as a solution in the field of cloud computing, especially in attempts to achieve balanced load distribution [7], [8], [9].

Like round-robin, priority scheduling is also widely used and its limitations are equally well defined. Numerous studies have been conducted to address the drawback of scheduling based on fixed priorities [10]. Similarly to round-robin, many studies in the field of cloud computing have explored the use of dynamic priority to overcome the limitations of traditional approaches [11], [12]. In the field of communication scheduling, dynamic priority adjustment has also been utilized [13], and there have been studies that applied dynamic priority to the FreeRTOS kernel, which is also discussed in this paper [14].

Furthermore, combining round-robin with priority scheduling has been a common approach to address its limitations [15]. The author in [16] attempted to address these issues using queuing theory, and there have also been efforts to synthesize and evaluate such approaches in a comprehensive way [17].

In the field of real-time systems, various dynamic scheduling approaches have been proposed to efficiently manage tasks on multicore platforms. Some studies focus on scheduling parallel real-time tasks, which can be modeled as directed acyclic graphs, and dynamically adjust priorities based on factors like memory or cache access to improve system performance [18], [19]. Other research explores dynamic scheduling for load balancing purposes, sometimes using agent-based systems that monitor queue lengths to distribute tasks, or by dispatching jobs to appropriate core types in heterogeneous systems [20], [21], [22].

Unlike prior works that predominantly focused on general-purpose systems with abundant resources, this paper targets the stringent constraints of embedded systems, demonstrating a practical implementation on resource-limited hardware. The proposed framework not only introduces a lightweight mechanism for dynamic scheduling, but also effectively integrates priority adjustments and round-robin scheduling into a unified model, achieving significant reductions in context switching overhead. Using an innovative feedback control mechanism to dynamically regulate average waiting time, this study ensures fairness and responsiveness, setting a new benchmark for task management in embedded environments.

In our previous work, dynamic round-robin scheduling was proposed; however, the method to determine priorities and the implementation of ultralight context switching were only introduced in a shallow manner [23]. Moreover, the previous approach utilized only two operating systems for switching, and the monitor mode was executed during every OS execution, resulting in higher context switching overhead from a system-wide perspective. In this study, we have proposed solutions to effectively address the aforementioned issues.

III. PROPOSED METHOD

This section begins by formally defining the problem that our research addresses in terms of its objectives and constraints. It then presents the proposed dynamic round-robin scheduling framework, detailing its core components: secure memory and functional isolation, a monitor mode for dynamic time-quantum adjustment, and an ultralight context switching mechanism.

A. PROBLEM FORMULATION

This subsection formally defines the problem of designing an adaptive real-time scheduling framework for resource-constrained embedded systems. The primary goal is to dynamically optimize task execution based on changing priorities while ensuring system fairness and operating within strict hardware limitations.

1) OBJECTIVES

The proposed scheduling framework is designed to achieve two primary objectives simultaneously:

- **Adaptive Performance for High-Priority Tasks:** The scheduler must dynamically allocate more CPU time to tasks whose priorities become high due to real-time events or changes in the external environment. The first objective is to minimize the execution time of high-priority tasks, thereby maximizing system responsiveness to critical events.
- **Starvation-Free Fairness for Low-Priority Tasks:** The framework must guarantee that low-priority tasks are not indefinitely postponed by high-priority tasks. This goal is achieved by ensuring that the AWT of all tasks converges towards a predefined target value, ensuring a baseline level of execution for every task.

2) CONSTRAINTS

The scheduler must operate under the following fundamental constraints:

- **Resource-Constrained Hardware:** The solution must be lightweight enough to run efficiently on single-core MCUs with limited Flash memory and SRAM, as detailed in Section V-A.
- **Minimal Scheduling Overhead:** The scheduling algorithm itself, including the operations of the monitor mode and the PID controller, must introduce minimal computational overhead to avoid impacting the overall system performance and to keep context switching times low.

3) APPLICATION MODEL

The framework is designed to manage a mixed workload of applications typical in advanced embedded systems. It assumes that the applications running on the platform consist of the following characteristics:

- **Periodic Tasks:** These are tasks that are executed on a recurring basis, such as continuous sensor monitoring or control loops. They form the baseline workload of the system.
- **Aperiodic Events:** The system must also respond to unpredictable events originating from external triggers or I/O interrupts. These events are not pre-scheduled and are responsible for the dynamic shifts in task priorities that our framework is designed to handle.
- **Soft Real-Time Requirements:** All tasks are considered to have soft real-time requirements. While timely execution is critical for responsiveness, the system prioritizes adaptation to priority changes over guaranteeing strict, deterministic deadlines for every task.

B. MEMORY AND FUNCTIONAL ISOLATION

In the context of this paper's multi-OS framework, ensuring the exclusive use of memory for each operating system is paramount. To this end, memory is strategically divided and allocated to each OS. This division takes into account the distinct characteristics of flash and SRAM, each of which plays a pivotal role in the performance of embedded systems due to their unique architecture, functionality, and application. By assigning each operating system to specific memory regions, our implementation promotes a secure and stable operating environment, ensuring the independence and integrity of each memory region.

Figure 2 illustrates how memory is allocated when using three operating systems and a monitor. The proposed memory structure, much like the Jailhouse hypervisor [24], enables resource allocation through complete isolation between operating systems. This isolation ensures robust system stability and security by preventing interference or unauthorized access between OS environments. In addition, the advantages of segregating memory spaces include improved fault tolerance, enhanced data integrity, and efficient resource

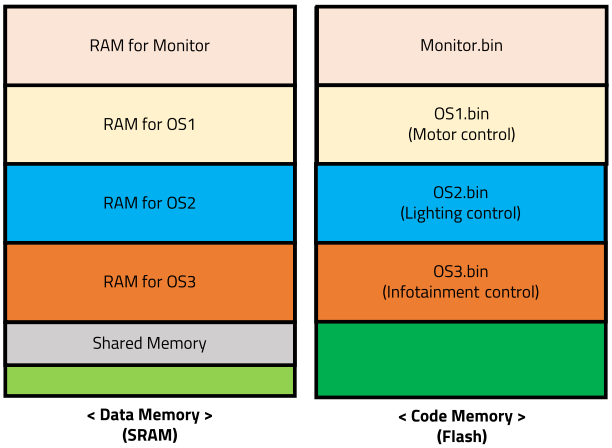


FIGURE 2. Examples of memory and functional isolation using three operating systems and monitor mode.

management. However, the size of each memory partition may need to be adjusted depending on the specific MCU used, reflecting the flexibility required for diverse hardware configurations.

To enforce this strict memory partitioning at a practical level, we customized the linker script for each operating system. This script explicitly defines the start and end addresses for the *.text* (code), *.data* (initialized data), and *.bss* (uninitialized data) sections, ensuring that each OS binary is confined exclusively to its pre-allocated memory region as shown in Figure 2. This compile-time enforcement prevents any possibility of memory overlap between OS instances and provides a robust, hardware-level foundation for memory isolation.

This paper proposes not only memory-level isolation between operating systems, but also functional isolation. Functional isolation within a hypervisor enhances system security and reliability by clearly separating the operational roles of each operating system. This separation ensures that the operating system can perform its specific functions without affecting others, reducing the likelihood of system-wide failures. In addition, functional isolation also simplifies troubleshooting and maintenance, as issues can be isolated to specific functional domains without impacting the entire system. By compartmentalizing functionalities, the hypervisor ensures a more organized and resilient embedded system architecture.

C. MONITOR MODE

As explained in the previous section, memory is fully isolated; however, inter-OS communication is necessary to evaluate characteristics such as the urgency or priority of each operating system. The monitor mode was designed to enable efficient resource management across the entire system and to determine the time-quantum in dynamic round-robin scheduling.

Figure 3 shows how the monitor mode contributes to the dynamic round-robin scheduling structure. A round is defined

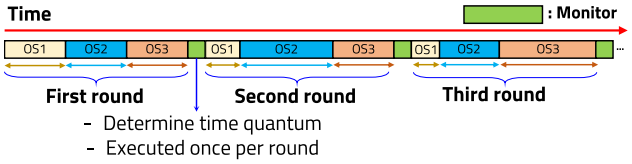


FIGURE 3. Introduction to the roles of the monitor mode.

as the completion of execution by all operating systems, each running for its allocated time-quantum. The monitor mode is executed before the start of each round to assess external environmental changes, such as sensor readings or voltage levels, and determine the priority and urgency of execution for each operating system.

Previous research ensured that monitor mode was executed during every context switching process between only two operating systems. As the research evolved to explore the use of more than two operating systems, executing monitor mode during every context switching was found to unnecessarily increase the CPU time dedicated to monitor mode, resulting in reduced overall efficiency. By reducing the execution of monitor mode to once per round, the overall system efficiency was improved, allowing the use of more diverse methods to assess external environmental conditions.

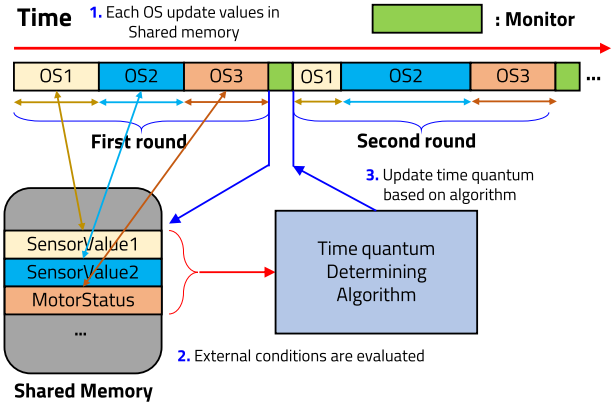


FIGURE 4. Time-quantum updating sequence by accessing shared memory.

Although each memory is isolated, the monitor mode must have access to each operating system's information to determine the time-quantum for its next run. Figure 4 illustrates the sequence of accessing shared memory to update the time-quantum. First, each operating system stores information about the external environment and its own urgency in the shared memory during execution. Clues about the external environment can be obtained from sensor data or I/O information, allowing the evaluation of each operating system's priority and urgency. Subsequently, the time-quantum is allocated to each operating system through the time-quantum determining algorithm, which will be discussed later. In the next round, each operating system is executed with the updated time-quantum.

While the monitor mode is essential for enabling dynamic scheduling, its execution represents a computational overhead not present in static schedulers. This overhead is a deliberate trade-off for gaining real-time adaptability. To mitigate its impact on overall system performance, our framework is designed to invoke the monitor mode only once per round, rather than on every context switch, striking a critical balance between responsiveness and computational cost.

D. ULTRALIGHT CONTEXT SWITCHING

Monitor mode does not exist in traditional round-robin scheduling and, from a system perspective, it can be regarded as an overhead. However, the functionality of monitor mode is crucial in dynamic round-robin scheduling, making it necessary to reduce overhead in other parts of the system to improve overall efficiency.

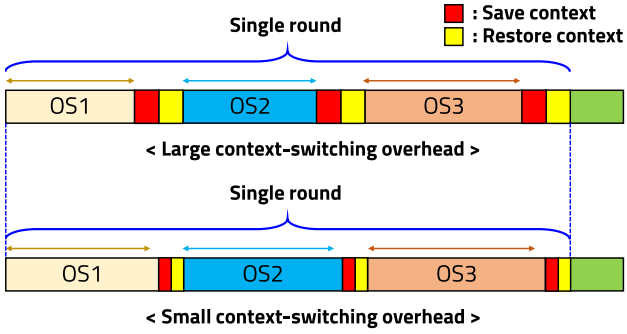


FIGURE 5. Impact of large context switching overhead.

Figure 5 illustrates how a significant context switching overhead can introduce inefficiencies into the overall system. Since no actual operations are performed during context switching, larger context switching overhead results in disadvantages both in terms of execution time and system responsiveness to external environments. In dynamic round-robin scheduling, which includes the additional overhead introduced by monitor mode, it became essential to minimize context switching overhead.

Previous research considered only the switching between two operating systems, and adjusting the time-quantum after the execution of all OS resulted in greater overhead. The current study reduced the overhead of dynamic round-robin scheduling across multiple operating systems by making it more lightweight. This result was achieved through functional isolation, which limits the roles of each operating system and reduces the number of registers used.

E. STARVATION-FREE FEEDBACK CONTROL

In the integration of priority-based scheduling with round-robin, the combined approach inherits both the starvation issue from priority scheduling and the context switching overhead from round-robin scheduling. However, the context switching overhead has been mitigated through the use of an ultralight context switching technique, as described previously. To address the starvation issue, this paper adopts a feedback control mechanism that uses the AWT instead of traditional aging techniques. While the integration with round-robin may help mitigate starvation, it introduces a trade-off: if the time-quantum allocated to each round is excessively large, the system's ability to respond promptly to changes in external conditions may be compromised. Since monitor mode is executed once per round and the time-quantum is updated at the same frequency, it is feasible to dynamically measure and regulate AWT for fair resource distribution. Equation (1) demonstrates the method for measuring AWT:

$$\text{AWT (Average Waiting Time)} = \frac{\sum_{i=1}^{n} W_i}{n} \tag{1}$$

where W_i denotes the time each OS waited before being allocated CPU time; n is the total number of OS.

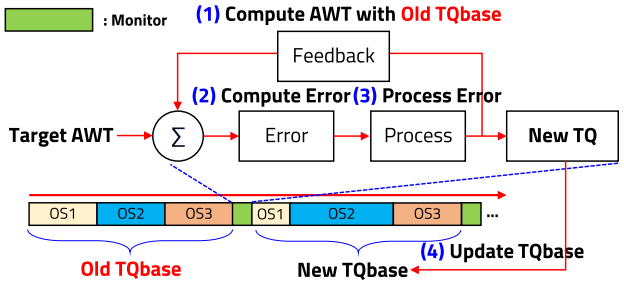


FIGURE 6. Diagram of AWT feedback control mechanism.

Figure 6 shows the sequence of the AWT feedback control mechanism. Upon entering monitor mode, the system calculates the AWT using the base time-quantum from the previous round, referred to as "Old TQbase" in the diagram. Subsequently, the error between the target AWT and the calculated AWT is determined, and the error value is appropriately processed to generate a new base time-quantum. The new base time-quantum becomes the time-quantum for the next round, and after the next round is completed, the same process is repeated. As will be discussed later, the determined base time-quantum is divided and distributed according to the priorities of each operating system.

For instance, to illustrate its practical operation, consider a scenario where the target AWT of the system is 100 ms, but the measured actual AWT after a round is 130 ms due to the dominance of high-priority tasks. Upon entering the monitor mode, the feedback controller identifies this 30 ms discrepancy and calculates a negative error. The PID logic processes this error to generate a positive corrective output, which is then added to the current base time-quantum. In the subsequent round, this slightly larger base time quantum provides longer execution slices for all tasks, helping bring the actual AWT back towards its 100 ms target. This continuous self-correcting loop ensures that even if low-priority tasks start to wait too long, the system

automatically adjusts to give them a fair opportunity to run, effectively preventing starvation.

Given the inherent resource constraints of MCU-based embedded systems, including limited computational capabilities and memory footprint, this research addresses hypervisor implementations where concurrent execution is bounded to a maximum of four to five operating systems. The limited number of concurrent operating systems inherently prevents extreme waiting-time distributions in this environment. This characteristic enables the AWT metric alone to serve as a sufficient indicator for starvation prevention, eliminating the need for additional metrics such as maximum waiting time or waiting time standard deviation.

The implementation of this PID-based feedback control introduces a calculable overhead during the monitor mode's execution. The primary cost stems from the time-quantum determination algorithm itself, which involves several arithmetic and floating-point operations for the PID formula, as detailed later in Section IV-A.

IV. IMPLEMENTATION

This section outlines the detailed implementation of our proposed dynamic round-robin scheduling framework. We present the memory isolation mechanism, the operation of the monitor mode, and the ultralight context switching methodology, each component designed to ensure efficient resource management while minimizing system overhead.

A. TIME-QUANTUM DETERMINING ALGORITHM

1) ASSESSMENT OF EXTERNAL CONDITIONS

As mentioned in the previous section, checking sensor values to assess external conditions is a priority when determining the time-quantum. This is related to the functional isolation of each operating system, ensuring that no OS performs overlapping functions with each other. This allows each OS to focus on its designated tasks and assess the current state based on sensor values and other inputs.

2) DETERMINING TIME-QUANTUM

The priority of the operating systems to be executed varies depending on the external conditions. Once the external condition is identified, priorities are assigned to each operating system based on the specific conditions. At this stage, the priority of each operating system must be determined either theoretically or heuristically through multiple iterations. These established priorities are then used in the algorithm to set the time-quantum. The time-quantum is computed as the product of the base time-quantum per round and the normalized priority ratio, where the priority ratio represents the individual priority value relative to the sum of all priorities. The formula (2) clearly shows this sequence.

$$TQ_i = TQ_{base}(t) \times \frac{P_i}{\sum P} \tag{2}$$

The system employs PID control mechanisms to regulate and maintain AWT at predetermined target levels. The error function, as shown in Equation (3), is derived from the difference between the target AWT and the mean observed AWT value. Based on this error, the PID controller generates a control output $u(t)$ as shown in Equation (4), where K_p, K_i, and K_d represent the proportional, integral, and derivative gains respectively.

$$e(t) = AWT_{avg}(t) - AWT_{target} \tag{3}$$

$$u(t) = K_p \cdot e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt} \tag{4}$$

This control output is then used to adjust the base time-quantum. As presented in Equation (5), the base time-quantum for the next round is determined by adding the control output to the current base time-quantum, while ensuring that the result remains within the predefined minimum and maximum bounds $TQ_{\min}$ and $TQ_{\max}$. Finally, Equation (6) illustrates how the actual time-quantum for each operating system is calculated by multiplying the regulated base time-quantum with the normalized priority ratio. This priority-weighted time-quantum allocation ensures an appropriate distribution of processor time while maintaining the desired average waiting time through PID control.

$$TQ_{\text{base}}(t+1) = \max(TQ_{\min}, \min(TQ_{\text{base}}(t)+u(t), TQ_{\max})) \tag{5}$$

$$TQ(i) = TQ_{\text{base}}(t) \times \frac{P_i}{\sum_{k=1}^{n} P_k} \tag{6}$$

3) ALLOCATING TIME-QUANTUM

The algorithm 1 describes how to update the time-quantum in monitor mode using the equations mentioned before. First, the proposed system initializes the sum of time-quantum values and AWT from allocated OS tasks and calculates the delta value for PID control. Next, it calculates the error terms of the AWT for the PID control. Using the calculated values, it computes a new base time-quantum and limits its range to prevent excessive fluctuations.

Then, the priorities for each OS must be set, and various sensor values can be utilized for this process. In this paper, the priority will be determined using the motor's revolutions per minute (RPM) value, and the method for determining the priority will be described in subsequent sections.

Finally, the algorithm concludes by dividing the base time-quantum according to the priorities to allocate to each OS, and updating the current values as previous values. Through PID-controlled time-quantum adjustment and priority-based distribution, this algorithm achieves adaptive scheduling that responds to real-time system requirements.

The structured sequence of algorithm is critical for its function. The initial steps in line 2 to 4 focus on measuring the system's current state(AWT), while steps in line 5 to 9 use this measurement to calculate the necessary temporal adjustment via the PID logic. This control decision is then translated into concrete scheduling parameters through the situational priority assessment in line 14 and the final time-quantum distribution in line 16-17. This clear separation of state

Algorithm 1 Update Time-Quantum Control

Function `UpdateTimeQuantum()`:
- `/* Step 1: Set initial values */`
- $tq\_sum \leftarrow \sum_{i=1}^{N} OS[i].tq$
- $awt \leftarrow (2 \times tq\_sum)/N$
- $delta\_t \leftarrow tq\_sum/1000.0$
- `/* Step 2: Error calculations */`
- $error \leftarrow AWT\_target - awt$
- $error\_integral \leftarrow error\_integral + (error \times delta\_t)$
- $error\_derivative \leftarrow (error - prev\_error)/delta\_t$
- `/* Step 3: PID controller */`
- $u \leftarrow (K_p \times error) + (K_i \times error\_integral) + (K_d \times error\_derivative)$
- $newTQ\_base \leftarrow currentTQ\_base + u$
- `/* Step 4: Limit TQ base value */`
- **if** $newTQ\_base < TQ\_min$ **then**
  - $newTQ\_base \leftarrow TQ\_min$
- **if** $newTQ\_base > TQ\_max$ **then**
  - $newTQ\_base \leftarrow TQ\_max$
- `/* Step 5: Set priorities */`
- `SetPriority(rpm)`
- `/* Step 6: Update time-quantum */`
- $priority\_sum \leftarrow \sum_{i=1}^{3} OS[i]\_priority$
- **for** $i \leftarrow 1$ **to** 3 **do**
  - $TQ[i] \leftarrow \lfloor (priority[i]/priority_{sum}) \times TQ_{base} \rfloor$
- `/* Step 7: Update past values */`
- **for** $i \leftarrow 1$ **to** 3 **do**
  - $temp[i] \leftarrow TQ[i]$
- $currentTQ\_base \leftarrow newTQ\_base$
- $prev\_error \leftarrow error$
- **return**

assessment, control decision, and parameter distribution ensures a modular and predictable control flow.

From a computational complexity perspective, this algorithm is designed to be highly efficient. The time complexity is $O(N)$, where N is the number of concurrent operating systems. This linear scaling arises because operations such as summing the initial time quanta and later distributing the new time quantum must iterate through all N OS instances. In contrast, the core PID logic for calculating the error and control output is a constant-time $O(1)$ operation, as its execution is independent of the number of operating systems. This predictable $O(N)$ behavior ensures that the scheduling overhead grows manageably as the system scales, reinforcing its suitability for resource-constrained environments.

B. ULTRALIGHT CONTEXT SWITCHING

As commonly implemented in numerous real-time operating systems (RTOS), this study used timer interrupts to facilitate round-robin scheduling implementation. However, since this study involves the implementation of multiple operating systems, each system requires its own time interrupt mechanism. The concurrent operation of heterogeneous interrupts inevitably generates system overhead; nevertheless, this challenge was effectively addressed through the strategic integration of multiple preexisting optimization techniques.

1) NESTED INTERRUPT HANDLING

When deploying multiple operating systems in embedded environments, a fundamental constraint arises from the inherent inability of a single-core board to process multiple interrupts concurrently, since embedded systems predominantly utilize single-core boards due to their cost effectiveness and power efficiency. Therefore, when a time-quantum ends, the board's vector table must be set as the vector table address of the OS to be executed next. Through this methodology, time-sharing characteristics can be implemented for interrupt handling employing separate vector tables for each operating system, thereby enabling efficient interrupt management.

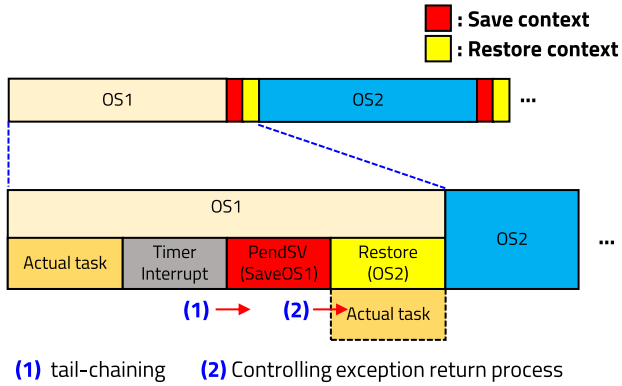


FIGURE 7. In-depth analysis of proposed context switching methodologies.

In conventional timer interrupt-based context switching systems, executing context switches directly within the timer's Interrupt Service Routine (ISR) presents a critical limitation as the ISR becomes excessively lengthy, compromising real-time performance and system interrupt latency. The implementation of PendSV (Pendable Supervisor Call) provides an optimal architectural solution wherein the timer interrupt solely triggers the PendSV bit while delegating the context switching mechanism to the PendSV exception handler. This approach maintains the timer interrupt's minimal service routine while utilizing PendSV's lowest exception priority, thus ensuring both interrupt responsiveness and atomic context switching operations.

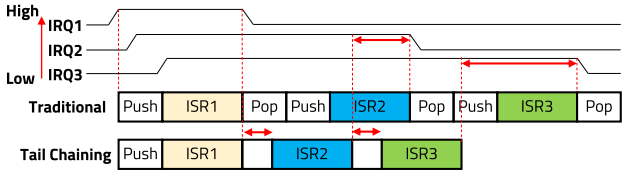


FIGURE 8. Brief examination of tail-chaining sequential operations.

The implementation of PendSV offers additional advantages. The scenario involved nested interrupts where timer interrupts and PendSV execute concurrently. As illustrated in Figure 8, conventional nested interrupt handling requires continuous stack operations to cause the occurrence of interrupt, resulting in degraded execution time and system responsiveness. However, the Cortex-M architecture employed in this research utilizes tail-chaining methodology, which significantly optimizes performance by eliminating redundant stack operations through the preservation of identical context information.

2) EXCEPTION RETURN

As described in the previous section, context switching is managed by the PendSV exception. Unlike regular functions in ARM architecture where the return process involves transferring the link register to the program counter to resume execution at the original location, exceptions such as PendSV undergo a specialized return process known as exception return.

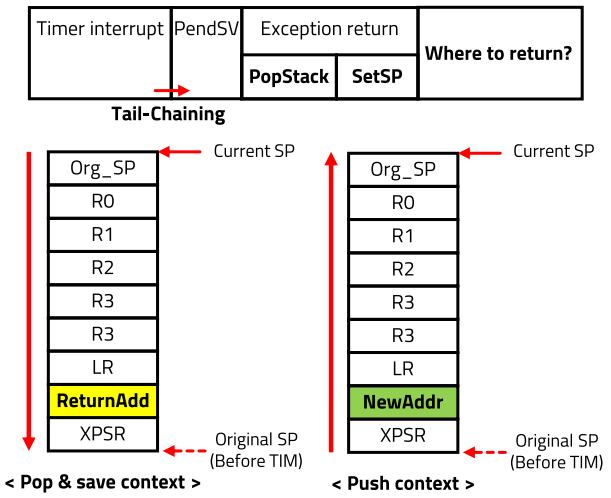


FIGURE 9. Exception return process in microcontroller unit.

Upon completion of PendSV execution, the hardware initiates an exception return sequence comprising stack restoration and stack pointer configuration. These hardware-driven processes preclude direct user intervention. The hardware manages the return sequence by orchestrating the required operational data and transforming them according to the user specifications. Figure 9 illustrates the sequential progression of the exception-return process. The procedure necessitates re-pushing intended values, in addition to the complete restoration of previously pushed stack values. A critical consideration is the configuration of the return address, which is predetermined as the initialization point for any function within the same operating system. If the return address is erroneously pushed to an alternate operating system's memory space, a fault condition occurs, as the post-stack-restoration pointer configuration inherently references the current memory domain.

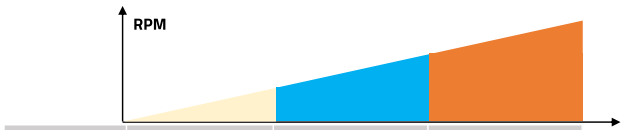
During the debugging phase, numerous errors emerged, predominantly stemming from inconsistent stack pointer values during inter-OS branches following exception returns. These issues were attributed to compiler-generated automatic stack push operations that occur during both exception handling and regular function execution. To address this challenge, the naked attribute was utilized for the exception handler. This compiler directive, which suppresses the automatic generation of function prologue and epilogue code, enables the direct user management of stack consistency. The naked attribute instructs the compiler to omit the usual automatic stack operations, allowing manual control over stack frame setup and cleanup operations. This solution proved essential for maintaining precise stack pointer configurations across operating system transitions, effectively preventing the stack corruption issues that had previously occurred during context switching.

V. EXPERIMENTAL SETUP AND EVALUATION

In this section, we evaluate the performance of dynamic round-robin and ultralight context switching by emulating hypervisor operations in vehicle driving scenarios.

A. EXPERIMENTAL ENVIRONMENT

The key aspect of our proposed dynamic round-robin method is that the time allocation to each OS adapts dynamically according to changes in external conditions. In this paper, our aim is to measure these changes by simulating vehicle driving scenarios. Fundamentally, the OS that becomes critical varies depending on the vehicle's current situation. Although various sensors can be used to assess surrounding conditions, in this experiment we use RPM values to determine the vehicle's current situation.



Role(OS)\Situation	Traffic Jam	Downtown	Highway
Motor (1)	High Priority	Mid Priority	Low Priority
Lighting(2)	Low Priority	Mid Priority	Low Priority
Infotainment(3)	High Priority	Low Priority	High Priority

FIGURE 10. Priority change in various driving situations.

As shown in the Figure 10, we classified scenarios and assigned priorities based on RPM values using a simple classification scheme. The use of multiple sensors would enable more granular scenario classification and refined priority assignment, which would be necessary for real-world deployment. Although we employ a simplified approach in this paper, the primary contribution lies not in the situation classification method but in demonstrating the system-wide advantages gained through dynamic round-robin scheduling.

The experimental scenario emulates vehicular operation conditions, in which the infotainment system represents a

computationally intensive task within the automotive environment. Therefore, we incorporated dummy computation tasks into OS3, responsible for infotainment operations, and primarily investigated the correlation between priority variation and task completion times.

Additionally, to evaluate the effectiveness of PID control in maintaining AWT proximity to the target value, the initial time-quantum allocation was configured at 100 ms for all operating systems at system initialization. This configuration establishes an initial AWT of 200ms, facilitating clear observation of the system's convergence behavior toward the target value. The PID control parameters were predetermined by empirical analysis. This empirical process involved iteratively running the test scenarios with different sets of K_p, K_i and K_d gains. We observed the system's response, focusing on the speed of convergence to the target AWT and the stability of the time-quantum adjustments. Through this iterative tuning, the gains for our automotive test scenarios were set to $K_p = 0.3$, $K_i = 0.001$, and $K_d = 0.05$. It is important to note that these optimal values are highly dependent on the specific characteristics of the tasks being executed. Therefore, a preliminary analysis and re-tuning of these parameters are considered essential steps before deploying the framework in a different real-world application to ensure stable and responsive performance.

TABLE 1. Specification of STM32F407G-DISC1.

MCU	STM32F407VGT6
Core	32-bit ARM Cortex-M4 with FPU core
Flash	1 Mbyte
RAM	192 Kbyte
Freq.	~169 MHz

To evaluate the proposed hypervisor architecture at the MCU level, we used the STM32F407G-DISC1 module based on STMicroelectronics' Cortex-M4 core, which has the specifications shown in Table 1. Although not featuring high-end performance specifications, this board was chosen to demonstrate that the proposed architecture can be successfully implemented even in environments with limited computational resources.

B. EXPERIMENTAL RESULTS

The experimental results are divided into two parts: a comparison of context switching overhead with existing methods and a comparison with the traditional round-robin approach.

1) CONTEXT SWITCHING OVERHEAD

To quantify the temporal benefits of the proposed method over conventional approaches, a comparative analysis of the context switching overhead was initially conducted. The reduction in context switching overhead directly contributes to system efficiency by minimizing the computational resources during OS transitions, thereby enabling more effective real-time task execution and improved response times in multi-OS environments.

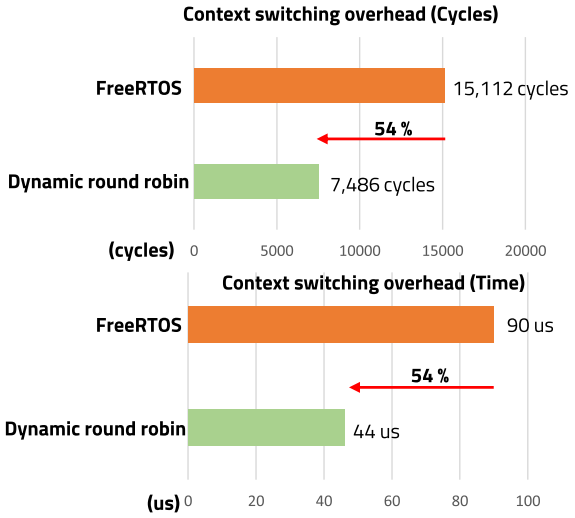


FIGURE 11. Context switching time comparison with FreeRTOS.

For comparative analysis, we used FreeRTOS, a prevalent RTOS solution in embedded systems, as a reference implementation. As shown in Figure 11, the proposed method exhibited a significant reduction in both context switching time and cycles, achieving an improvement of 54% over FreeRTOS. This substantial reduction in context switching overhead significantly contributes to system performance by minimizing CPU overhead during OS transitions, thereby enabling more efficient task scheduling and improved real-time response capabilities in multi-OS environments.

2) COMPARISON WITH TRADITIONAL METHOD

This section demonstrates the scheduling behavior of the proposed method, specifically examining how OS priorities dynamically adjust to RPM variations, as outlined in the experimental environment section.

First, Figure 12 (a) illustrates the average time quanta assigned to each OS. The result demonstrates a uniform time-quantum allocation across all OS, exhibiting constant AWT values throughout the execution period. Figure 12 (b) shows that the observed dummy computation completion time of 3.34 seconds serves as a reference point for comparative analysis with subsequent scheduling scenarios. In the conventional approach, all AWTs are identical; therefore, a target AWT was not specified. Instead, the experiment was conducted with an AWT set to 200 ms.

Figure 13 (a) shows the average time quanta assigned to each OS under traffic jam conditions. The allocation of a larger time-quantum to OS1 and OS3, corresponding to their elevated priorities. Figure 13 (b) shows the results in a dummy computation completion time of 2.67 seconds. With the target AWT set to 300 ms, the originally configured AWT of 200 ms was adjusted through feedback control, resulting in an average scheduled AWT of 286 ms.

Target AWT	Actual Avr. AWT	Avr. OS1 TQ	Avr. OS2 TQ	Avr.OS3 TQ
-	200ms	100ms	100ms	100ms

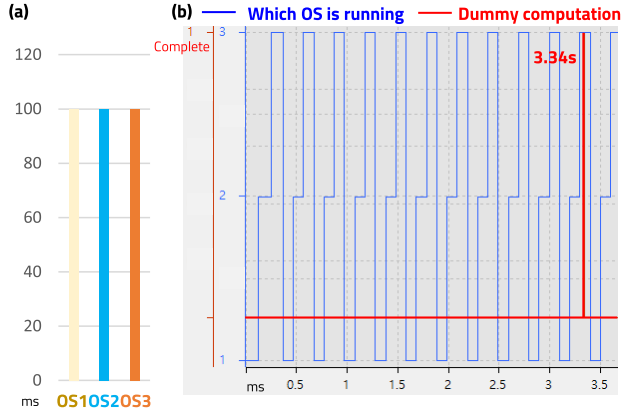


FIGURE 12. Experimental result for traditional round-robin (a) Time allocated to each OS (b) State of CPU allocation and end point of the dummy computation.

Target AWT	Actual Avr. AWT	Avr. OS1 TQ (Mid Priority)	Avr. OS2 TQ (Mid Priority)	Avr.OS3 TQ (Low Priority)
300ms	284ms	150ms	150ms	75ms

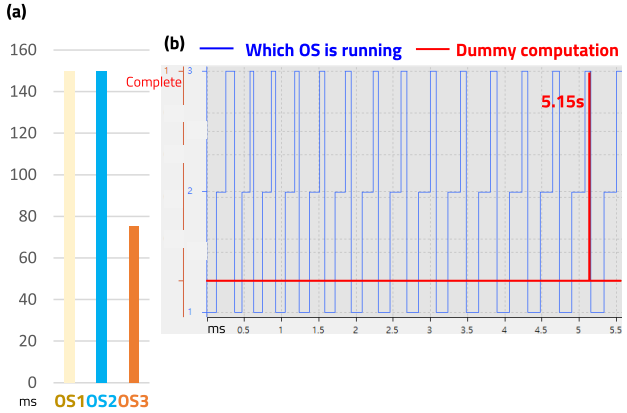


FIGURE 14. Experimental result for scenario 2(Downtown) (a) Time allocated to each OS (b) State of CPU allocation and end point of the dummy computation.

Target AWT	Actual Avr. AWT	Avr. OS1 TQ (High Priority)	Avr. OS2 TQ (Low Priority)	Avr.OS3 TQ (High Priority)
300ms	286ms	165ms	110ms	165ms

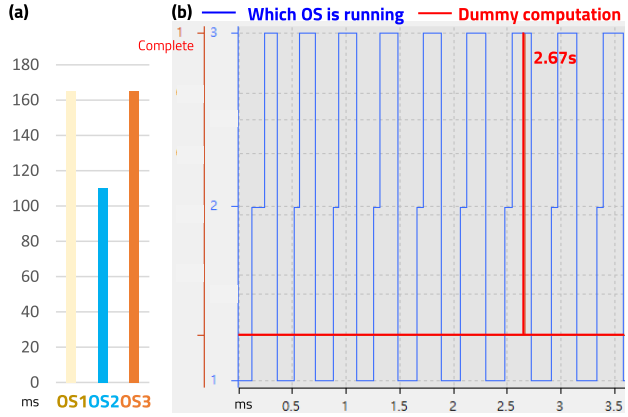


FIGURE 13. Experimental result for scenario 1(Traffic Jam) (a) Time allocated to each OS (b) State of CPU allocation and end point of the dummy computation.

Figure 14 (a) presents the average time-quantum simulated for vehicular operation under downtown driving conditions. In this scenario, the time-quantum allocated to OS3 decreases proportionally to its decreased priority level. As a result, Figure 14 (b) shows that the completion time of the dummy computation increased to 5.15 seconds. Similarly, with the target AWT set to 300 ms, the originally configured AWT of 200 ms was adjusted through feedback control, resulting in an average scheduled AWT of 284 ms.

Although the computation completion time exhibits an increase, this outcome is particularly significant as it validates the system's capability to effectively defer the execution of lower-priority tasks. As both priority and time-quantum are inherently relative metrics among operating systems, the observed delay in less critical task execution indicates enhanced execution capabilities for higher priority tasks, demonstrating the system's effective resource allocation mechanisms. The significant contribution of this work is to provide users with unprecedented control over task execution by enabling lazy execution of less critical tasks.

Target AWT	Actual Avr. AWT	Avr. OS1 TQ (Low Priority)	Avr. OS2 TQ (Low Priority)	Avr.OS3 TQ (High Priority)
300ms	266ms	68ms	68ms	206ms

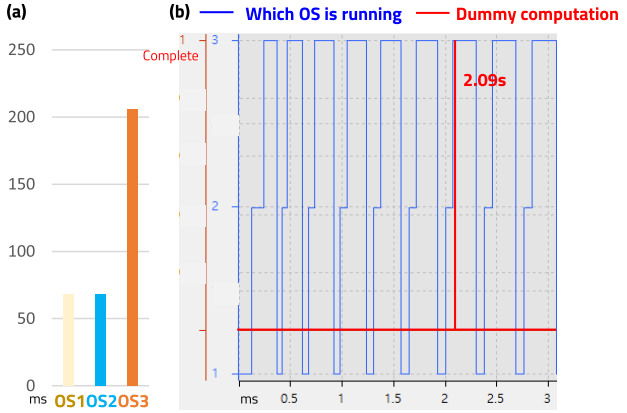
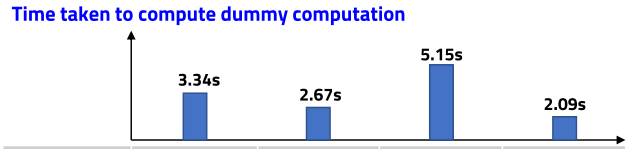


FIGURE 15. Experimental result for scenario 3(Highway) (a) Time allocated to each OS (b) State of CPU allocation and end point of the dummy computation.

Figure 15 (a) presents the results of the scheduling performance under driving conditions on the road. This scenario effectively demonstrates the relative nature of time-quantum allocation: while OS3 maintained high priority in the traffic jam scenario, it achieved its shortest dummy computation completion time of 2.09 seconds in this scenario due to the lower priorities of other OSs, as shown in Figure 15 (b).



Role(OS)\Situation	Traditional	Traffic Jam	Downtown	Highway
Motor(1)	Mid Priority	High Priority	Mid Priority	Low Priority
Lighting(2)	Mid Priority	Low Priority	Mid Priority	Low Priority
Infotainment(3)	Mid Priority	High Priority	Low Priority	High Priority

FIGURE 16. Experimental result for scenario 3 (Highway) (a) Time allocated to each OS (b) State of CPU allocation and end point of the dummy computation.

Finally, Figure 16 presents a comprehensive summary of the results obtained by performing dummy computations in the four scenarios described above. It is evident that as the priority of OS3, which handles infotainment, increases, the time required to complete the dummy computation decreases. In the traffic jam scenario, the OSs responsible for infotainment and motor have higher priority. As a result, the dummy computation in the infotainment system took approximately 20% less time than it did with the traditional method. Additionally, in the Downtown scenario, due to the low priority of Infotainment, the time required for dummy computation increased by approximately 54% compared to the traditional method. In the highway scenario, since only the priority of infotainment is set to High, the time required for dummy computation is reduced by 38% compared to the traditional method, demonstrating that scheduling can focus more on specific tasks.

The experimental results demonstrate that providing users with the ability to modify function priorities in an event-driven manner contributes to creating an adaptive system. Additionally, all experimental scenarios demonstrated convergence of the AWT to the target values through PID control implementation, indicating the system's ability to maintain consistent responsiveness levels. The comprehensive experimental evaluation supports the efficiency of the proposed scheduling methodology, demonstrating both significant reduction in context switching overhead and adaptive resource allocation capabilities, while maintaining consistent system responsiveness through PID-controlled AWT management across diverse operational scenarios.

Regarding the scalability of the proposed framework for larger and more complex real-world systems, the linear computational complexity($O(N)$) of our scheduling algorithm provides a strong foundation for predictable performance. As the number of concurrent operating systems increases, the scheduling overhead grows manageably, not exponentially. Furthermore, the ultralight context switching mechanism, proven to be 54% more efficient than a standard RTOS, becomes even more advantageous in larger systems with frequent task switching. However, scaling to a significantly larger number of tasks would introduce new challenges, such as the increased complexity of tuning PID parameters, which may require more advanced auto-tuning techniques.

VI. CONCLUSION

This paper presented a novel PID-controlled dynamic round-robin scheduling framework with ultralight context switching for resource-constrained embedded systems. The proposed framework addresses several key challenges in modern MCU-based systems: efficient task prioritization, dynamic resource allocation, and minimal context switching overhead.

Our experimental results demonstrate significant improvements over traditional approaches in three key areas. First, the framework achieves a 54% reduction in context switching overhead compared to FreeRTOS, requiring only 7,486 cycles (44 us) for context switching operations. Second, the PID-controlled AWT management successfully maintains system responsiveness under varying operational conditions, as evidenced by consistent convergence to target values in all test scenarios. Third, the dynamic priority adjustment mechanism effectively allocates resources based on real-teim conditions, demonstrated by execution time variations from 2.09 to 5.15 seconds depending on task priorities.

Although we validated the framework through automotive use cases, where it successfully managed multiple operating systems with varying priorities based on driving conditions, its applicability extends far beyond this domain. For example, in industrial automation systems, manufacturing process priorities can dynamically shift based on production line sensors and real-time quality control requirements. In healthcare monitoring devices, task priorities could be adapted based on patient vital signs, ensuring critical health data processing takes precedence when needed. Smart building management systems could leverage this framework to balance HVAC control, security monitoring, and energy optimization tasks based on occupancy patterns and environmental conditions.

The ultralight context switching mechanism and memory isolation architecture ensure efficient operation even on modest hardware platforms, as demonstrated by our implementation on an STM32F407 MCU. This versatility makes our framework a practical solution for various resource-constrained environments requiring dynamic task management.

A particularly promising direction for future research is the integration of deadline-based scheduling with our priority-based approach. By combining the dynamic priority adjustment capabilities of our current framework with deadline scheduling mechanisms, we aim to develop a more comprehensive scheduler that can better address hard real-time constraints while maintaining the benefits of adaptive resource allocation. This hybrid approach would be especially valuable for systems that require both dynamic adaptability and strict timing guarantees.

This research contributes to the field by bridging the gap between traditional static scheduling methods and the dynamic requirements of modern embedded systems, providing a practical solution that balances efficiency, flexibility, and reliability in diverse application domains.

REFERENCES

[1] S. S. Thiebaut, A. De Rosa, and R. Sasse, "Secure embedded hypervisor based systems for automotive," in *Proc. 46th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. Workshop (DSN-W)*, Jun. 2016, pp. 211–212.

[2] K. Lampka and A. Lackorzynski, "Using hypervisor technology for safe and secure deployment of high-performance multicore platforms in future vehicles," in *Proc. 26th IEEE Int. Conf. Electron., Circuits Syst. (ICECS)*, Nov. 2019, pp. 783–786.

[3] M. U. Farooq, A. Shakoor, and A. B. Siddique, "An efficient dynamic round Robin algorithm for CPU scheduling," in *Proc. Int. Conf. Commun., Comput. Digit. Syst. (C-CODE)*, Mar. 2017, pp. 244–248.

[4] R. Mishra and G. Mitawa, "Improved round Robin algorithm for effective scheduling process for CPU," in *Proc. 3rd Int. Conf. Intell. Commun. Technol. Virtual Mobile Netw. (ICICV)*, Feb. 2021, pp. 590–593.

[5] R. Sumathi and R. M. Savithramma, "Maximizing CPU performance: Advancing efficiency and fairness through adaptive round Robin scheduling with triple queues (ARRTQ)," in *Proc. 2nd Int. Conf. Data Sci. Inf. Syst. (ICDSIS)*, May 2024, pp. 1–6.

[6] H. Hu, H. Xu, Y. Luo, Y. Meng, S. Huang, and J. Huang, "Dynamic event-triggered control under the round-Robin protocol for 2-D continuous system in Roesser model," in *Proc. China Autom. Congr. (CAC)*, 2023, pp. 89–94.

[7] S. Ghosh and C. Banerjee, "Dynamic time quantum priority based round Robin for load balancing in cloud environment," in *Proc. 4th Int. Conf. Res. Comput. Intell. Commun. Netw. (ICRCICN)*, Nov. 2018, pp. 33–37.

[8] A. Katangur, S. Akkaladevi, and S. Vivekanandhan, "Priority weighted round Robin algorithm for load balancing in the cloud," in *Proc. IEEE 7th Int. Conf. Smart Cloud (SmartCloud)*, Oct. 2022, pp. 230–235.

[9] P. Niranjan and M. Thenmozhi, "Priority-induced round-Robin scheduling algorithm with dynamic time quantum for cloud computing environment," in *Proc. 2nd Int. Conf. Adv. Comput. Intell. Commun. (ICACIC)*, Dec. 2023, pp. 1–5.

[10] S. Zhang, J. Li, Q. Yang, and K. S. Kwak, "Dynamic priority scheduling mechanism based on spatio-temporal correlation for VANETs," in *Proc.12th Int. Conf. Ubiquitous Future Netw. (ICUFN)*, 2021, pp. 115–119.

[11] G. M. Karthik, A. Gupta, S. Rajeshgupta, A. Jha, A. Sivasangari, and B. P. Mishra, "Efficient task scheduling in cloud environment based on dynamic priority and optimized technique," in *Proc. Int. Conf. Artif. Intell. Smart Commun. (AISC)*, Jan. 2023, pp. 1–6.

[12] Y. Bo, L. Feng, and Z. Xiaoyu, "Cloud computing task scheduling algorithm based on dynamic priority," in *Proc. IEEE 6th Inf. Technol. Mechatronics Eng. Conf. (ITOEC)*, vol. 6, Mar. 2022, pp. 1696–1700.

[13] C. Hu, W. Wang, Y. Cai, H. Xue, Y. Kang, and Z. Cai, "Communication scheduling strategy for the power distribution and consumption Internet of Things based on dynamic priority," in *Proc. 5th Int. Conf. Renew. Energy Power Eng. (REPE)*, Sep. 2022, pp. 50–55.

[14] K. Salamun, I. Pavic, and H. Dzapo, "Dynamic priority assignment in FreeRTOS kernel for improving performance metrics," in *Proc. 44th Int. Conv. Inf., Commun. Electron. Technol. (MIPRO)*, Sep. 2021, pp. 880–885.

[15] S. Ullah, "Improved optimum dynamic time slicing round Robin algorithm," in *Proc. 3rd Int. Conf. Electr. Inf. Commun. Technol. (EICT)*, Dec. 2017, pp. 1–6.

[16] M. A. Mohammed, M. AbdulMajid, B. A. Mustafa, and R. F. Ghani, "Queueing theory study of round Robin versus priority dynamic quantum time round Robin scheduling algorithms," in *Proc. 4th Int. Conf. Softw. Eng. Comput. Syst. (ICSECS)*, Aug. 2015, pp. 189–194.

[17] A. A. Alsulami, Q. A. Al-Haija, M. I. Thanoon, and Q. Mao, "Performance evaluation of dynamic round Robin algorithms for CPU scheduling," in *Proc. SoutheastCon*, Apr. 2019, pp. 1–5.

[18] L. Zhenyang, L. Xiangdong, and L. Jun, "Scheduling parallel real-time tasks for multicore systems," in *Proc. Int. Conf. Service Sci. (ICSS)*, Aug. 2020, pp. 102–106.

[19] T. Shimizu, H. Nishikawa, X. Kong, and H. Tomiyama, "A non-work conserving algorithm for dynamic scheduling of moldable gang tasks on multicore systems," in *Proc. Int. Conf. Electron., Inf., Commun. (ICEIC)*, Jan. 2024, pp. 1–4.

[20] M. Kim, Y. Soo, and J.-W. Jeon, "Multicore ECU task-load distribution (balancing) and dynamic scheduling," in *Proc. IEEE Region 10 Symp. (TENSYMP)*, 2021, pp. 1–5.

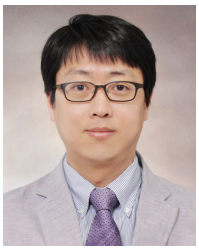
[21] K. Baital and A. Chakrabarti, "Various approaches for high throughput and energy efficient scheduling of real-time tasks in multicore systems," in *Proc. IEEE Int. Symp. Smart Electron. Syst.*, Dec. 2019, pp. 402–405.

[22] G. Muneeswari and S. R. Reeja, "Agent based queue aware scheduling for distributed multicore system," in *Proc. IEEE 7th Int. Conf. Recent Adv. Innov. Eng. (ICRAIE)*, vol. 7, Dec. 2022, pp. 1–6.

[23] M. Kim and D. Park, "Implementation of dynamic round Robin scheduling on bare-metal shallow multi-OS for lightweighted microcontrollers," in *Proc. IEEE 48th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, Jul. 2024, pp. 1556–1557.

[24] H. Yang, J. Zhang, R. Shao, Y. Chen, R. Zhou, and Q. Zhou, "RJMM: Real-time enhancement for jailhouse hypervisor on multi-core platforms via memory isolation," in *Proc. 9th Int. Conf. Dependable Syst. Appl. (DSA)*, 2022, pp. 498–502.

MINJUNG KIM (Student Member, IEEE) received the bachelor's degree in electronic engineering from Kyungpook University, Daegu, South Korea, in 2024. He is currently pursuing the M.S. degree with the School of Electronic and Electrical Engineering, Kyungpook National University, Daegu. His main interest is intermittent computing, with a strong focus on scheduling for lightweight microcontrollers and security. He has extensive experience in developing algorithms for resource-constrained environments and optimizing system performance, and has published several articles in these fields. He is researching technologies to optimize deep learning-based object detection algorithms to be applied to low-power embedded systems. He plans to explore lightweight AI (on-device AI) for embedded systems in the future.



DAEJIN PARK (Member, IEEE) received the B.S. degree in electronics engineering from Kyungpook National University (KNU), Daegu, South Korea, in 2001, and the M.S. and Ph.D. degrees in electrical engineering from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea, in 2003 and 2014, respectively. He has been a Full Professor of electronics engineering (EE) with KNU, since 2014. He was a Research Engineer with SK Hynix Semiconductor, Samsung Electronics more than 12 years, from 2003 to 2014, and has worked on designing low-power embedded processors architecture and implementing a fully AI-integrated system-on-chip with intelligent embedded software on the custom-designed hardware accelerator, especially for hardware/software tightly coupled applications, such as smart mobile devices and industrial electronics. He has published more than 250 technical articles and 60 patents. He was nominated as one of the Presidential Research Fellows 21, Republic of Korea, in 2014. He was a recipient of Korea Prime Minister Award as one of the major contributors to the AI-embedded system-software-on-chip technology innovation, in 2023.